

AY: 2025-2026

EXAM | ROS 2

Dec. 2025

L3-S5: Dept. of Electrical Engineering

Teacher: A. Mhamdi

Time Limit: 1½ h

This document contains 8 pages numbered from 1 to 8. Upon receiving it, verify completeness. The 4 tasks are independent and can be solved in any order you prefer. The following rules apply:

- ❶ A handwritten double-sided A4 sheet is permitted.
- ❷ Any electronic material, except basic calculator, is prohibited.
- ❸ Mysterious or unsupported answers will not receive full credit.
- ❹ Round results to the nearest thousandth (i.e., the third digit after the decimal point).
- ❺ Task N°4: Correct answers earn points as indicated. There is no negative scoring.

Task N°1

⌚ 15mn | (3½ points)

Write exactly the command(s) you would type in a terminal.

- (a) (½ point) Create a new ROS 2 workspace called `ros2_ws` in your home directory, and add a subfolder named `src` to it.

```
mkdir -p ~/ros2_ws/src
```

- (b) (½ point) Navigate into the directory `ros2_ws`.

```
cd ~/ros2_ws
```

- (c) (½ point) Create a Python package named `sensor_pkg` that depends on `rclpy` and `std_msgs`.

```
ros2 pkg create --build-type ament_python sensor_pkg
→ --dependencies rclpy std_msgs
```

- (d) (½ point) Build the (still-empty) workspace with `colcon`.

```
colcon build --symlink-install
```

- (e) ($\frac{1}{2}$ point) Source the newly built workspace so you can use its packages.

```
source ~/ros2_ws/install/setup.bash
```

- (f) ($\frac{1}{2}$ point) After creation, colcon will read setup.py. What is the main purpose of setup.py in a Python ROS 2 package? (≤ 15 words)

Specifies how colcon should install the package, its scripts, and its entry points.

- (g) ($\frac{1}{2}$ point) You add a node script src/sensor_pkg/sensor_pkg/imu_node.py. To make it executable from the command line as `ros2 run sensor_pkg imu_node`, what exact line must be added inside the entry_points dictionary in setup.py?

```
'imu_node = sensor_pkg.imu_node:main'
```

Task N°2

⌚ 15mn | (3 points)

Your robot uses differential drive and publishes velocity commands on `/cmd_vel` (type: `geometry_msgs/msg/Twist`). Your task is to make it follow a circular path with radius $r = 2$ meters and forward speed $v = 1$ m/s. The Twist message structure is:

```
1 geometry_msgs/msg/Vector3 linear
2   float64 x # forward/backward
3   float64 y # left/right (usually 0)
4   float64 z # up/down (usually 0)
5 geometry_msgs/msg/Vector3 angular
6   float64 x # roll (usually 0)
7   float64 y # pitch (usually 0)
8   float64 z # yaw (rotation about vertical)
```

- (a) (1 point) Compute angular velocity ω

$$v = r \times \omega \quad \therefore \quad \omega = \frac{v}{r}$$

The angular velocity is 0.5 rad/s.

- (b) (1 point) List all 6 Twist values (*linear.x, y, z; angular.x, y, z*).

The twist values are:

```
{ linear.x = 1.0
  linear.y = 0.0
  linear.z = 0.0
  angular.x = 0.0
  angular.y = 0.0
  angular.z = 0.5 }
```

(c) ($\frac{1}{2}$ point) Write the full **ROS 2** topic pub command to send one message to `/cmd_vel`

```
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist
↪ "{linear: {x: 1.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y:
↪ 0.0, z: 0.5}}"
```

(d) ($\frac{1}{2}$ point) Write the command to subscribe and view `/cmd_vel` messages live.

```
ros2 topic echo /cmd_vel
```

Task N°3

⌚ 30mn | (6 points)

You created a custom message in package `robot_interfaces`:

```
1 # File: robot_interfaces/msg/DriveCommand.msg
2 float64 linear_velocity      # m/s
3 float64 angular_velocity    # rad/s
4 bool    emergency_stop      # true = stop immediately
```

You built the workspace and sourced it. Write a Python subscriber node that:

1. Subscribes to `/drive_cmd` (type: `robot_interfaces/msg/DriveCommand`)
2. If `emergency_stop == True`, print: **EMERGENCY STOP ACTIVATED!**
3. Otherwise, print: Moving: `v = X.XX m/s, w = X.XX rad/s`

```
1 import rclpy
2 from rclpy.node import Node
3 from robot_interfaces.msg import DriveCommand
4
5
```

```

6 class DriveSubscriber(Node):
7     def __init__(self):
8         super().__init__('drive_subscriber')
9         self.subscription = self.create_subscription(
10             DriveCommand,
11             '/drive_cmd',
12             self.callback,
13             10 # QoS depth
14         )
15         self.get_logger().info('Drive subscriber started.')
16
17     def callback(self, msg):
18         if msg.emergency_stop:
19             self.get_logger().error('EMERGENCY STOP ACTIVATED!')
20         else:
21             self.get_logger().info(
22                 f'Moving: v = {msg.linear_velocity:.2f} m/s, '
23                 f'w = {msg.angular_velocity:.2f} rad/s')
24
25
26 def main(args=None):
27     rclpy.init(args=args)
28     node = DriveSubscriber()
29     try:
30         rclpy.spin(node)
31     except KeyboardInterrupt:
32         pass
33     finally:
34         node.destroy_node()
35         rclpy.shutdown()
36
37 if __name__ == '__main__':
38     main()

```



Terminal

```

student@isetbz:~\ros2_ws$ colcon build --packages-select robot_interfaces my_robot
student@isetbz:~\ros2_ws$ source install/setup.bash
student@isetbz:~\ros2_ws$ ros2 run my_robot drive_subscriber

```

AY: 2025-2026

L3-S5: Dept. of Electrical Engineering

EXAM | ROS 2

Dec. 2025

Teacher: A. Mhamdi

Full Name:

ID:

Class: ELNI3

Room:

Time Limit: 1½ h

ANSWER SHEET

Task N°4

⌚ 30mn | (7½ points)

(a) (½ point) What does the pwd command display?

- ☐ The current user's home directory
- ☐ The parent directory of the current directory
- ☒ The current working directory
- ☐ The list of all environment variables

(b) (½ point) What is the result of this bc command?

```
1 echo "scale=2; 22 / 7" | bc
```

3.14

(c) (½ point) Write the output of this for loop?

```
1 for i in $(seq 1 2 5); do
2 echo $i
3 done
```

1
3
5

(d) (½ point) What is the purpose of the self keyword in a class method in Python?

- ☐ Refers to the class itself
- ☒ Refers to the instance of the class
- ☐ Refers to the parent class
- ☐ Refers to global variables

(e) (½ point) What does the code print?

✂

```
1 class A:
2     def greet(self):
3         print("Hello from A")
4 class B(A):
5     def greet(self):
6         super().greet()
7         print("Hello from B")
8 b = B()
9 b.greet()
```

- ☐ Only "Hello from B"
- ☒ "Hello from A" followed by "Hello from B"
- ☐ Error in "super()"
- ☐ "Hello from A"

(f) ($\frac{1}{2}$ point) What is the primary function of a node in ROS 2?

- ☐ Store sensor data in a database
- ☐ Compile Python code for robot control
- ☒ Perform computations and communicate with other nodes via topics or services
- ☐ Manage hardware drivers exclusively

(g) ($\frac{1}{2}$ point) Which command is used to list all active ROS 2 nodes in a system?

- ☐ ros2 topic list
- ☐ ros2 service list
- ☒ ros2 node list
- ☐ ros2 param list

(h) ($\frac{1}{2}$ point) What is the purpose of a topic in ROS 2?

- ☐ To execute Python scripts
- ☐ To store configuration parameters
- ☒ To allow nodes to exchange messages in a publish-subscribe model
- ☐ To manage node lifecycle

(i) ($\frac{1}{2}$ point) Which message type is commonly used for sending velocity commands to a robot in ROS 2?

✂

- ☐ std_msgs/msg/String
- ☒ geometry_msgs/msg/Twist
- ☐ sensor_msgs/msg/Image
- ☐ nav_msgs/msg/Odometry

(j) ($\frac{1}{2}$ point) What does the following **ROS 2** command do?

```
ros2 topic echo /cmd_vel
```

- ☐ Publishes a message to the /cmd_vel topic
- ☐ Lists all messages published on the /cmd_vel topic
- ☒ Subscribes to and prints messages from the /cmd_vel topic
- ☐ Creates a new topic named /cmd_vel

(k) ($\frac{1}{2}$ point) In a **ROS 2** subscriber node, what is the purpose of a callback function?

- ☐ To initialize the node
- ☒ To process incoming messages from a subscribed topic
- ☐ To publish messages to a topic
- ☐ To set up the **ROS 2** middleware

(l) ($\frac{1}{2}$ point) Consider the following Python code for a **ROS 2** publisher node:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.publisher_ = self.create_publisher(String, '/chatter', 10)
        self.timer = self.create_timer(1.0, self.timer_callback)
    def timer_callback(self):
        msg = String()
        msg.data = 'Hello, ROS 2!'
        self.publisher_.publish(msg)
```

What is the name of the topic this node publishes to?

- ☐ /talker

✂

- ☒ /chatter
- ☐ /std_msgs
- ☐ /hello

(m) ($\frac{1}{2}$ point) Consider this Python code for a **ROS 2** subscriber node:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32
class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        self.subscription = self.create_subscription(
            Int32, 'counter', self.callback, 10)
    def callback(self, msg):
        self.get_logger().info(f'Received: {msg.data}')
```

What happens when a message is received on the counter topic?

- ☐ The node publishes the message to another topic
- ☒ The node logs the integer value of the message
- ☐ The node shuts down
- ☐ The node creates a new topic

(n) ($\frac{1}{2}$ point) Which of the following is a valid **ROS 2** message type for representing a 2D or 3D point in space?

- ☐ std_msgs/msg/Float64
- ☐ sensor_msgs/msg/LaserScan
- ☒ geometry_msgs/msg/Point
- ☐ action_msgs/msg/GoalStatus

(o) ($\frac{1}{2}$ point) What is the purpose of the `rclpy.spin()` function in a **ROS 2** Python node?

- ☐ To initialize the **ROS 2** node
- ☒ To keep the node running and processing callbacks
- ☐ To publish messages at a fixed rate
- ☐ To shut down the node gracefully